What Else Needs Fixing?

EMNLP2026 BUDAPEST

NTT

Exploring Cost-Effective Test-Time Compute for Revision Propagation in Artifacts Generated Through Conversation

Daisuke Kikuta NTT, Inc.

Blog Code

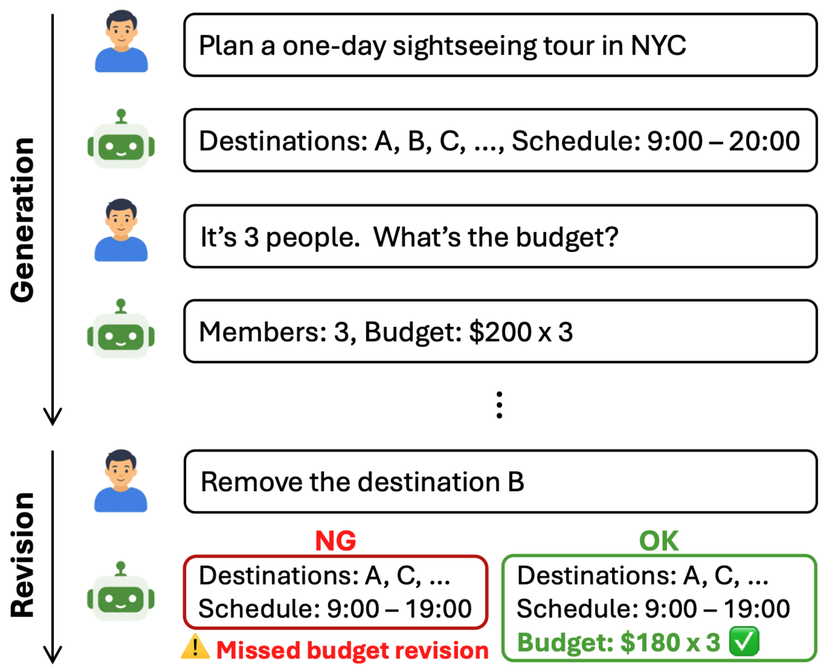
1. Introduction

Background

- LLMs often help users generate artifacts through iterative cycles of generation and revision in conversation
- **When users specify only a local change, LLMs must identify dependencies and update all affected parts**

Contributions

- We study this ability of LLMs on conversationally generated artifacts by **proposing a new benchmark**
- Toward practical use, we also **explore cost-effective test-time compute for this problem**



2. Proposed Benchmark

Objective

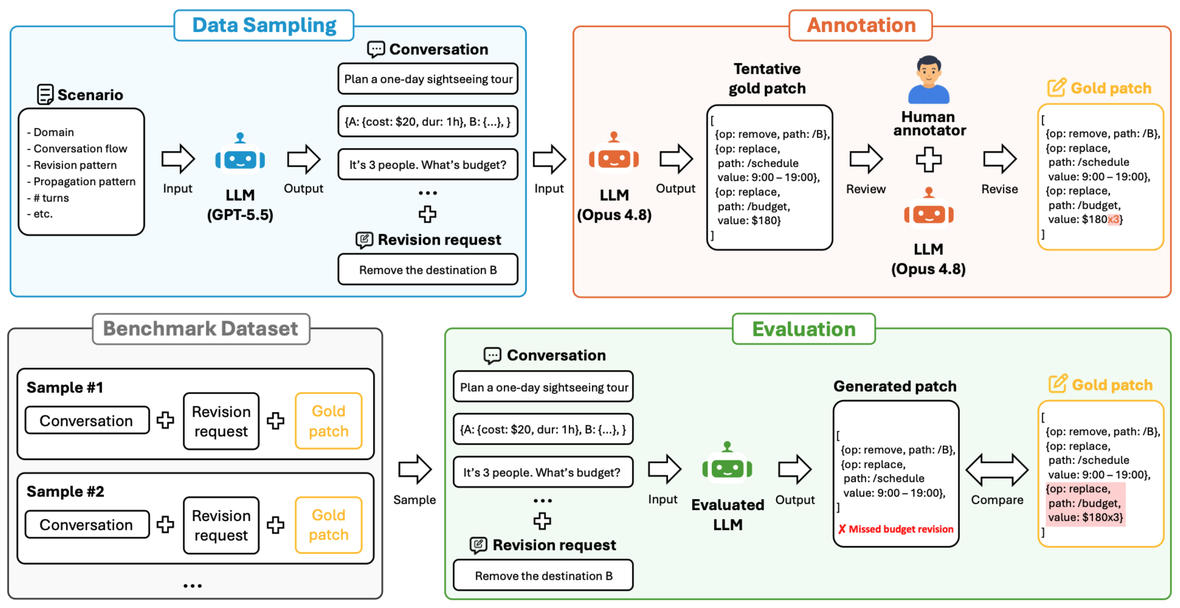
- RevPropBench evaluates whether LLMs can patch all and only the dependent parts of JSON artifacts, given the conversation history and local revision request

Data generation

- We synthetically generate conversation-request pairs with controlled gold-label determinism and dependency depth
- Each pair is labeled through LLM-assisted human annotation

Statistics

- 150 samples (dev: 30, test: 120), 9 practical domains, 6 propagation patterns, 3 artifact sizes (10/50/100 elems)



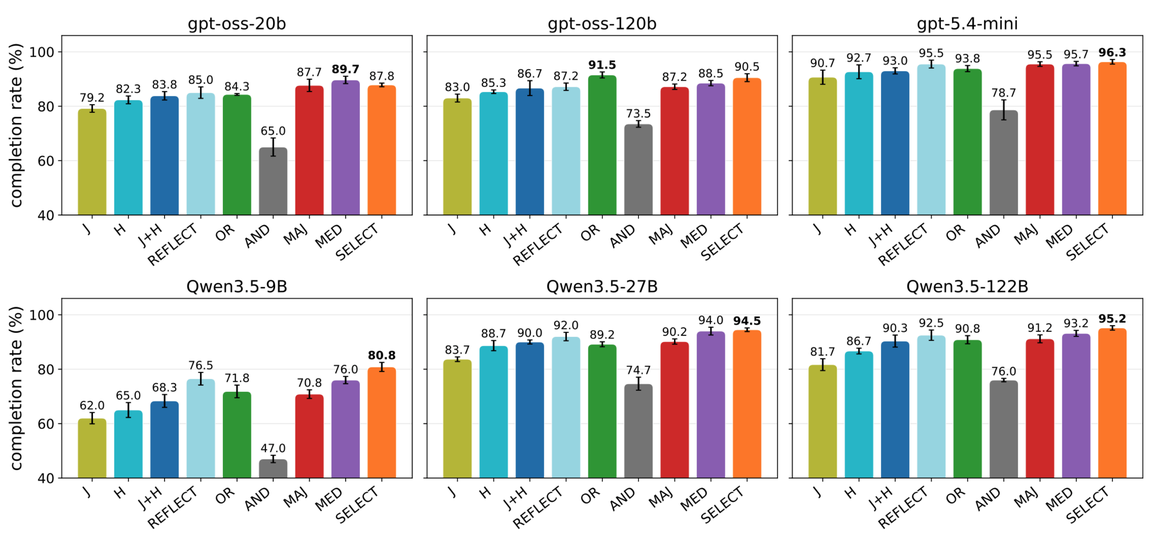
3. Benchmark Results

Evaluation settings

- **Evaluated models:**
  gpt-oss-20b/120b, gpt-5.4-mini, qwen3.5-9b/27b/122b
- **Evaluated revision methods:**
  Baselines (history only / JSON only / both in context),
  Sampling with rule-based merge (majority/or/and/medoid),
  Sampling with LLM-based selection, Sequential reflection
  *Sampling methods use 5 LLM calls

Key findings

- Baseline methods achieve accuracies of 68.3-93%
- Conversation history is critical for capturing conversationally established dependencies and element semantics (J<H<J+H)
- Sampling with medoid/LLM-based selection improves accuracy the most over the baselines (by 1.8-12.5pp)



4. Cost Analysis

Practical question

- **What is the most cost-effective number of samples for the two best-performing methods?**
- We investigate this by extending the number of samples and measuring how performance scales

Key findings

- **3 samples are the most cost-effective** for both methods
  *LLM-based selection uses 4 LLM calls (3 samples + 1 selection)
- LLM-based selection increases latency by ~2–5x.
  If latency matters, medoid offers a better trade-off, with only ~1.4x latency and ~1.4pp lower performance

